BUSINESS & TECHNOLOGY REPORT · AGENTIC AI PROGRAMME

Meridian AI

From four inboxes to one decision memo

How procurement checks that previously took days are converted by an agentic-AI platform into an auditable, policy-grounded recommendation in about two minutes. The final decision is still made by people.

~2 min
A full four-step audit is completed in about two minutes, replacing days of hand-offs (observed 67–150 s)

5 / 5
Policy questions were answered correctly in a known-answer test, including an honest "I don't know"

3 / 3
Audit scenarios reached the expected decision: approve, hold and reject

Case: **Aldermoor Industries** (fictional German machinery maker)

October 2026
Prepared by Maheep

EXECUTIVE SUMMARY

What was built, and why it matters

Meridian AI was built as a working, end-to-end agentic-AI application for **procurement intelligence**. Policy questions are answered from the company's own documents. Every purchase request is audited by several AI specialists before payment. One memo is produced, with a clear decision and a full audit trail.

The business problem

- **Slow:** each purchase is held in four inboxes in turn (Risk, Tax, Control, CFO), so days are lost.
- **Hidden rules:** payment terms, approval limits and tax rules are buried in long PDFs.
- **Inconsistent:** different answers to the same question are given by different buyers.
- **No trail:** when an auditor asks why a purchase was approved, no answer can be given.

The solution

- **Document helper (RAG):** questions are asked in plain English and answered only from company policy, with the source named.
- **Audit helper (multi-agent):** each request is checked by three AI specialists (Risk, Tax, Control) using tools. Written decision rules are then applied in a CFO step.
- **Guardrails:** personal data is redacted, cost and loop limits are enforced, failures are retried, and every tool call is logged.

What was accomplished

Dimension	Accomplishment
Speed	A full audit (3 specialists, 5 tools, CFO synthesis) is completed in about 70–150 seconds. A policy question is answered in about 4 seconds.
Consistency & control	The final decision is governed by **explicit rules**: RED ALERT → REJECTED; FX ALERT / TREASURY HOLD → CONDITIONAL HOLD; all clear → APPROVED TO PAY
Accuracy evidence	All 5 expected answers were matched in known-answer tests (e.g. revenue EUR 2.84 bn, Net-45 terms, 2.2% duty). The expected decision was reached in all 3 audit scenarios.
Auditability	Every phase and tool call is recorded in structured JSON logs, with arguments, duration and outcome. Fixed layouts are used for all reports.
Deployability	The app is moved from a laptop (Gemini key only) to Google Cloud (Cloud Run, Vertex AI Vector Search, Cloud Storage, Secret Manager) by a single `APP_MODE` setting. No code changes are required.
Engineering quality	A layered, modular codebase of about 1,900 lines of Python was delivered with a React UI. 17 automated tests are run offline, with no accounts required.

Design principle: "the system prepares, people decide." Research, checks and the write-up are performed by the AI. Authority over approvals, exceptions and policy is retained by people. Adoption in a regulated finance function is made possible by this principle.

Report map

1	Business context and problem	p. 3–4	5	Functional flow of the code	p. 13–15
2	Business solution and value	p. 5–6	6	Quality, operations and cost	p. 16
3	Technical solution	p. 7	7	Limits and roadmap	p. 17
4	Technical architecture	p. 8–12	8	Key takeaways, appendix	p. 18–19

1 · BUSINESS CONTEXT

A company that buys before it builds

Aldermoor Industries (fictional) is a Hamburg-based maker of filling and packaging machinery with more than 40 subsidiaries. Servo motors, PLC controllers, sensors, steel and chemicals are bought from suppliers around the world. Every purchase must be cleared by four checks **before** payment is released.

Risk & Compliance

Is the supplier permitted (sanctions)? Can delivery and solvency be relied on (credit)?

Tax & Treasury

How much VAT and import duty is due? Is the quoted exchange rate fair?

Finance & Control

Is it an asset (CapEx) or a running cost (OpEx)? At what level must it be approved?

CFO

With everything weighed: is the purchase paid, held or rejected?

Current state: one order, four inboxes

SEQUENTIAL · MANUAL · HARD TO TRACE

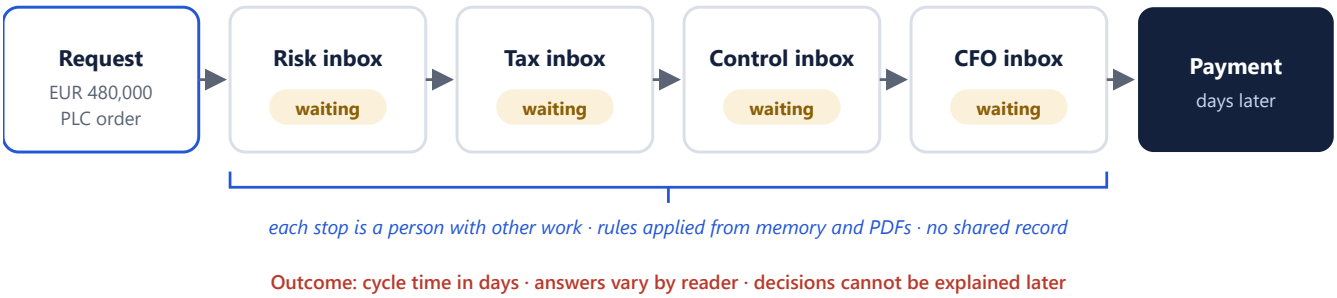


Figure 1. Current process: four sequential manual hand-offs are required before payment.

Four pains, one process

Pain	What happens	Business consequence
It is slow	Each department's check is performed in turn, and the request is left waiting in an inbox.	A EUR 480k order is delayed for days by three sign-offs. Supplier relationships and production schedules are affected.
The rules are hidden	Payment terms, approval limits and tariff rules are held in long PDFs.	To answer "What are the terms for servo-motor suppliers?", the right page in the right file has to be found.
People differ	"Net-30, I think" is given by one buyer; "Net-45 for a credit score of 75 or more" by another.	Policy is applied unevenly, and working-capital leakage and compliance exposure are created.
There is no record	The reasons for a decision are not written down in one place.	An auditor's "why was this approved?" cannot be answered, leaving a governance gap.

1 · THE PROBLEM, SIZED

Speed alone is not enough. A miss is expensive.

A sanctioned supplier is paid

Legal and regulatory exposure is created for the company and its managers.

An asset is booked as a running cost

The balance sheet and depreciation charge are misstated, and EBITDA is distorted.

An off-market FX rate is accepted

More is paid than the goods are worth, and treasury exposure is left unhedged.

No trail is kept

A decision cannot be explained when it is questioned by an auditor or regulator.

These consequences are illustrative, not statistics. They show why correctness and explainability are required, not just speed.

Why the obvious fixes fall short

Option considered	What is fixed	What is left open
Reminder e-mails / SLAs	A little speed is gained	Rules stay hidden, and different answers are still given by different people.
Shared folder of PDFs	Rules are made findable	The rules still have to be read and applied correctly by someone, every time.
A general chatbot	Fast, fluent answers are given	Aldermoor's documents have never been seen by it, and sanctions lists or FX rates cannot be checked. Confident but generic answers are produced.
Hard-coded rules engine	Consistency is achieved	PDFs cannot be read and supplier risk cannot be judged. Every policy change has to be made by IT.

The insight: an AI model on its own can be compared to a capable new hire on day one. The company's files have **never been seen**, and **no-one can be phoned**. Two remedies are combined. Through **retrieval**, the documents are supplied. Through **agents with tools**, facts are looked up and calculations are performed.

Requirements defined for the solution

1. **Speed.** Checks are completed in minutes, not days.
2. **Grounding.** Answers are drawn from Aldermoor's own rules.
3. **Consistency.** The same input is given the same decision.
4. **Record.** Every step can be explained afterwards.
5. **Human control.** Outputs are prepared; decisions are made by people.
6. **Honesty.** "I don't know" is given rather than an invention.

Stakeholders and how the system is used

Persona	Job to be done	Where in the app
Buyer	A purchase request is submitted and a decision memo is received for sign-off	Procurement Audit tab
Policy owner / admin	Policy, tariff, accounting and annual-report PDFs are published once	Index Documents tab
Any employee	Questions such as "What does the policy say about…?" are asked in plain English	RAG Q&A tab
IT / operations	Health, mode, models and logs are monitored	System Status tab and JSON logs

Two helpers, one application

The operating model is kept, but the waiting is removed. Ownership of the rules is retained by each specialist function. The rules are applied by the AI in support of a person's judgement, not in place of it.

① Document helper: "ask the policy"

Policy PDFs are uploaded once by an administrator. Questions can then be asked by anyone, and answers are **taken only from those documents**, with the source file attached. When the answer is not in the documents, this is stated.

```
Q: Payment terms for servo-motor suppliers?
A: Net-45 for suppliers with a credit score ≥ 75.
```

② Audit helper: "check this purchase"

A purchase request is pasted in by a buyer. It is investigated by three AI specialists, each with its own tools, and a report is written by each in a fixed layout. All three are read in a CFO step, and **one memo with a decision, reasons and next actions** is produced for human sign-off.

Future state: minutes, not days

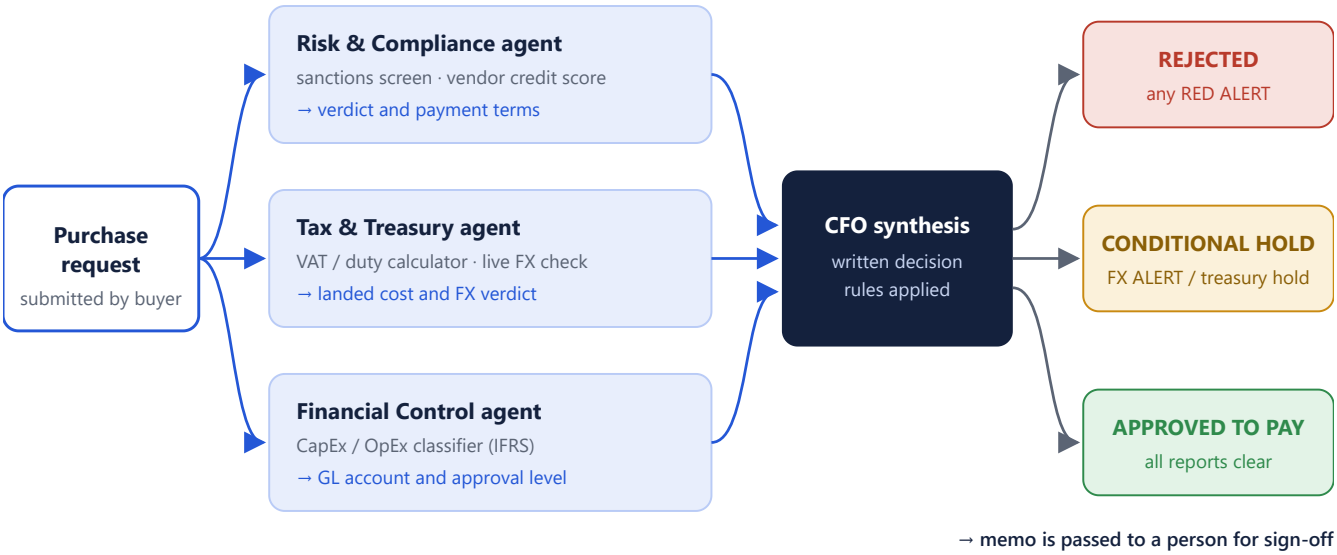


Figure 2. Target process: three specialist checks feed one rule-based CFO memo. Precedence: Reject → Hold → Approve.

A day in the life of a buyer

Time	What happens
Morning	Four PDFs are uploaded by the administrator: annual report FY2024, procurement policy, EU tariff excerpt and IFRS accounting policy. They are indexed by meaning.
10:00	Payment terms for servo-motor suppliers are asked about. **Net-45** (credit score ≥ 75) is returned in about 4 seconds, from the procurement policy.
11:00	An order for 200 PLC controllers (EUR 480,000) from Takumi Controls Europe B.V. is submitted (made in Japan, delivered to Germany), quoted at 1 EUR = 163.5 JPY.
~11:02	The memo is returned: CONDITIONAL HOLD. The quoted FX rate is found to be more than 5% off market, the supplier is unknown (high credit risk), and Department Head sign-off is required (over EUR 250k). Next actions are assigned to named teams.

Business value, measured, not assumed

Value was framed around four levers: **cycle time, consistency, traceability and self-serve policy access**. One rule was set for the pilot: **the baseline is measured first**. With a baseline, the gain becomes a number owned by the business.

Evidence 1: known-answer test of the document helper

Question asked	Expected	Observed	Result
Total revenue in FY2024 (annual report)	EUR 2.84 billion	EUR 2.84 billion	match
Payment terms, servo-motor suppliers (procurement policy)	Net-45	Net-45	match
Customs duty, PLC controllers from Japan (EU tariff)	2.2%	2.2%	match
Depreciation, filling-line equipment (IFRS policy)	10 years	10 years, straight line	match
"Who won the 2018 World Cup?" (not in documents)	I don't know	I don't know	match

Evidence 2: three audits, three correct decisions

Purchase request	Specialist findings	Decision	Run time
20 spare servo motors, EUR 38,000, well-known Japanese supplier, rate within band	Approved by Risk, Tax and Control	APPROVED TO PAY	~70 s
200 PLC controllers, EUR 480,000, unknown supplier, quoted 163.5 JPY vs ~177 market	FX ALERT raised; high credit risk; above EUR 250k	CONDITIONAL HOLD	~88 s
Precision steel, EUR 150,000, from a sanctioned Russian entity	RED ALERT raised on sanctions; credit check skipped, as instructed	REJECTED	~96 s

Observed in test runs, 9–10 Oct 2026 (Gemini flash, local mode). Wording varies between runs; layout and decision words do not.

> **The most valuable finding was a disagreement.** A customs duty of **0%** was estimated by the audit's tax tool from general AI knowledge. A duty of **2.2%** is stated in the company's own tariff document, as found by the document helper. Connection of the audit to authoritative sources has therefore been placed first on the roadmap.

KPI framework for a live pilot

KPI	Definition	Target direction
Request-to-payment cycle time	Days from submission to payment release	Reduced sharply (baseline first)
Hand-offs and rework	Requests returned to the buyer; inbox hops	Reduced
Decision agreement rate	Share of AI recommendations accepted by the human approver	High and stable; disagreements reviewed
Explainability coverage	Share of decisions with a full memo and log trail	100%
Cost per audit	Model tokens plus infrastructure, per request	Tracked; capped by call limits

Every component is traced to a business need

Each technology choice was justified by the need it serves. The architecture can therefore be explained to a CFO as easily as to an engineer.

Business need	Capability	Technology used
Access from a browser	Four-tab web interface	React + TypeScript (Vite, Tailwind, shadcn/ui)
Screen safely connected to logic	REST API with validated contracts	FastAPI, Pydantic schemas, Uvicorn
Answers drawn from company documents	Retrieval-Augmented Generation (RAG)	LangChain v1 (LCEL), PyPDF loader, recursive chunking
Text found by meaning, not keywords	Embeddings + vector search	Gemini embeddings (768-dim); Chroma locally or Vertex AI Vector Search in the cloud
Reasoning and writing in business English	Large language model	Google Gemini (flash model, temperature 0 for repeatable output)
A purchase checked from several angles	Multi-agent system with tools	LangChain `create_agent` (on LangGraph); 3 specialists, 5 tools, live Frankfurter FX API
Decisions governed by written rules	Rule-based synthesis	CFO prompt with fixed decision keywords and precedence
Data protected, cost capped, failures survived	Agent guardrails, secret management	LangChain middleware (PII redaction, call limits, retries, tool-call log); keys held in `.env` or Secret Manager
Run anywhere, scaled on demand	Container and serverless hosting	Docker (two-stage build), Google Cloud Run (scale to zero, max 3 instances)
Changes delivered without manual work	CI/CD and provisioning	GitHub Actions running idempotent `gcloud` steps
Faults diagnosed quickly	Observability	structlog JSON logs (Cloud Logging compatible), flushed to Cloud Storage on shutdown

Four key design decisions

Specialists over one "super-agent"

Each agent is given a short, focused brief and only the tools it needs, so reliability and auditability are improved. The trade-off is accepted: more model calls are made, so more time and cost are incurred.

Plain, readable orchestration

Risk → Tax → Control are run in a fixed order by a simple supervisor, followed by one CFO call. A predictable, debuggable flow is obtained. Less flexibility than a self-organising graph was deliberately accepted.

Outcomes decided by rules

The final verdict is determined by keyword rules (RED ALERT, FX ALERT, TREASURY HOLD). The reasoning is drafted by the AI, but the outcome is set by policy.

One switch from laptop to cloud

Storage and vector database are swapped behind one interface by `APP_MODE=local|cloud` (strategy pattern). Pilots can be started at near-zero cost, and production is run on managed services.

Logical architecture: four layers, one direction

Each layer is permitted to call only the layer below it. Nothing about AI is known to the web layer, and nothing about web addresses is known to the AI code. Any part can therefore be changed, tested or replaced on its own.

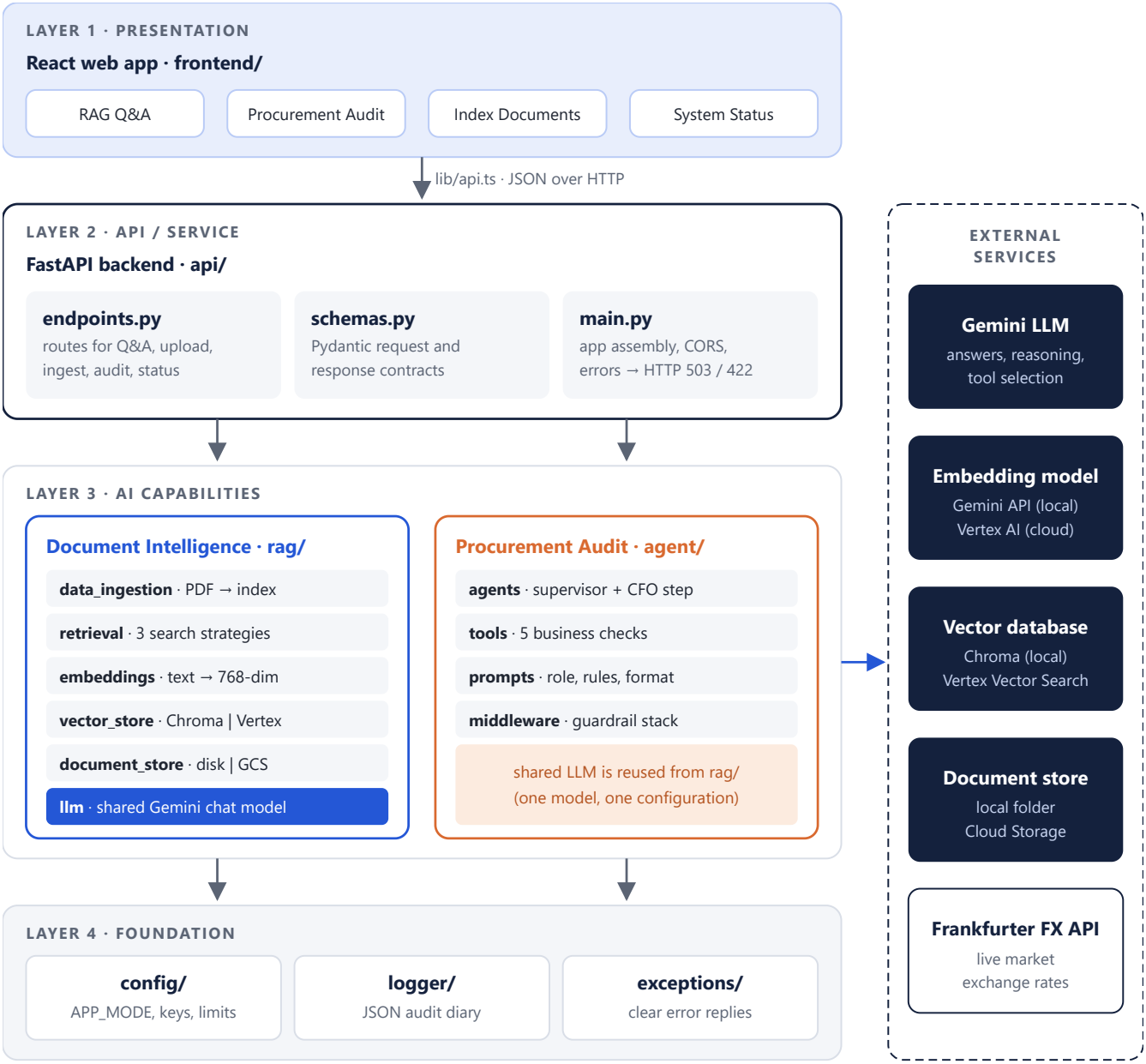


Figure 3. Logical architecture: packages, responsibilities and external dependencies. Calls flow downward only (api → rag, agent → config, logger, exceptions); external services are reached only from Layer 3.

Deployment architecture: piloted locally, run in the cloud

The same code is run in two modes, selected by one setting. The cost of proving value is thereby lowered: a pilot can be run locally with only a model key, and managed Google Cloud services are adopted once the case has been made.

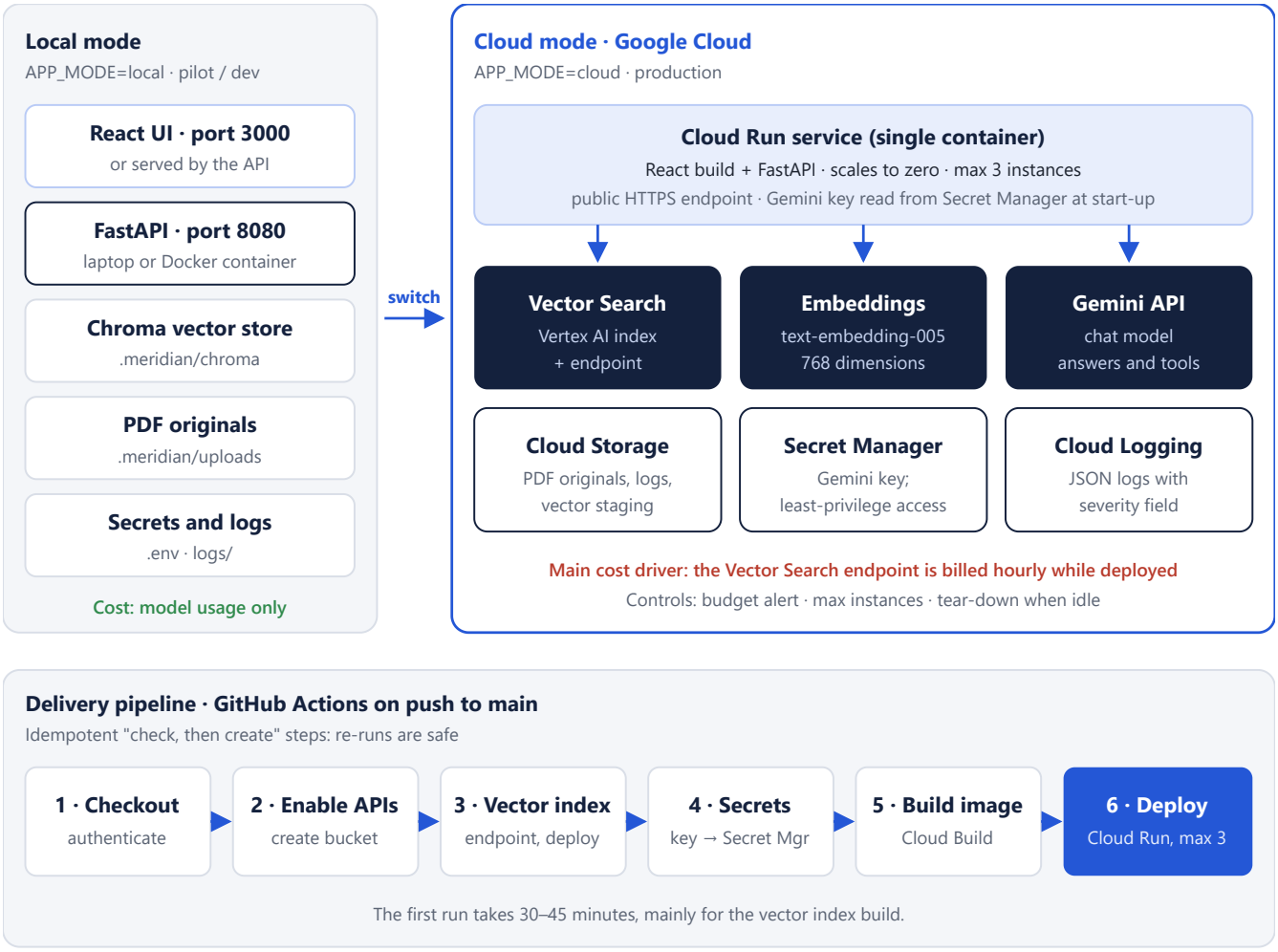


Figure 4. Local and cloud deployment topologies and the automated delivery pipeline (as specified in the project's deployment design).

Concern	Local mode	Cloud mode
Embeddings	gemini-embedding-001 via API key	text-embedding-005 via Vertex AI
Vector database	Chroma (folder on disk)	Vertex AI Vector Search (managed; scales to millions of chunks)
Files and logs	Local folders	Cloud Storage bucket (logs copied there on shutdown)

4 · ARCHITECTURE DEEP DIVE: DOCUMENT INTELLIGENCE

RAG: an open-book exam for the AI

In Retrieval-Augmented Generation, the right pages are found first and the answer is written only from them. The documents are not "remembered" by the model; they are looked up for each question and supplied as evidence.

STAGE A · INDEXING (ONCE PER DOCUMENT) · POST /api/rag/upload



STAGE B · ANSWERING (EVERY QUESTION) · POST /api/rag/ask



The same embedding model is used for storing and for searching.

THREE SELECTABLE RETRIEVAL STRATEGIES (QUALITY VS. SPEED AND COST)

Similarity (default)
Top-3 nearest chunks are used.
Fastest and cheapest: 1 model call.
Suited to direct policy look-ups.

Multi-query
The question is rephrased 3 ways;
all 4 are searched and merged.
Recall is improved for vague questions.

Contextual compression
10 chunks are fetched and trimmed
to relevant sentences. Sharper context
is obtained at a higher cost.

Figure 5. RAG pipeline: indexing and answering stages, with the three retrieval strategies that can be selected by the user.

Why these settings were chosen (business translation)

Choice	Why it matters to the business
"Answer only from context, otherwise say you don't know"	An honest "I don't know" is preferred to a confident invention, especially for policy and compliance questions.
Source name attached to every chunk	Answers can be traced back to a file and verified by anyone.
1,000-character chunks with 100 overlap	One complete rule is carried by each piece, and sentences on chunk borders are not lost.
Stable chunk IDs (hash of source, page, position)	When an updated policy is re-uploaded, the old version is overwritten rather than duplicated.
Scanned (image-only) PDFs skipped with a clear reason	Users are informed immediately. OCR has been placed on the roadmap.

A digital audit committee: three specialists and a CFO

An **agent** is an AI model that is permitted to **use tools** to obtain facts before answering. A tool is chosen, its result is read, and the cycle is repeated until the report can be written. Each specialist is modelled on a real department, with its own brief, tools and output format.

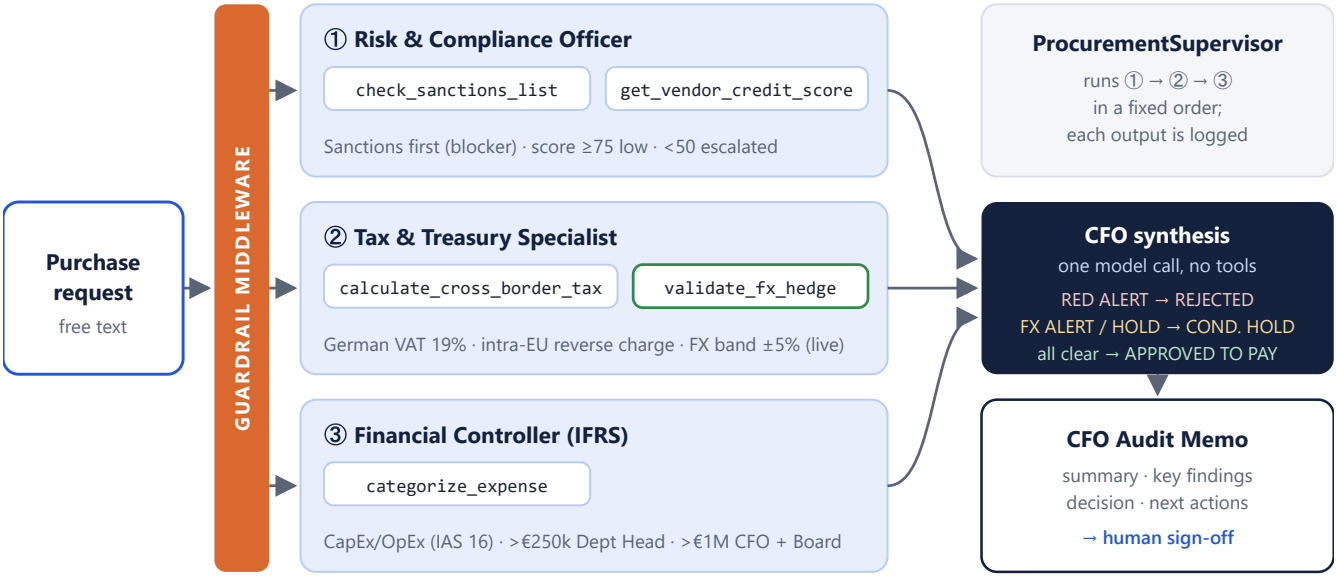


Figure 6. Multi-agent audit: specialists with dedicated tools are wrapped in guardrails, and their reports are fed into a rule-based CFO synthesis. The green tool uses live market data.

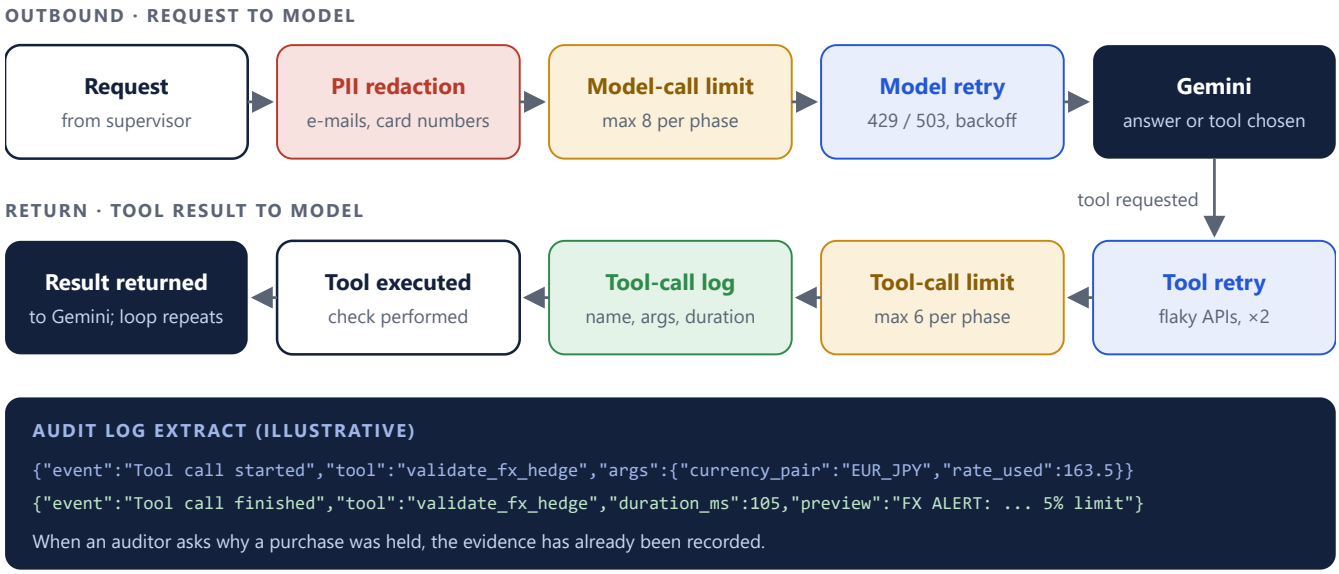
The five tools: what is checked and how failures are handled safely

Tool	Business check	Data source today → fail-safe behaviour
check_sanctions_list	OFAC, EU, UN and UK sanctions screening	Model knowledge (stand-in) → "SYSTEM WARNING, manual review required" is returned if unavailable
get_vendor_credit_score	Supplier financial health, 0–100	Model knowledge; a cautious score of 45 is given to unknown vendors → manual review on failure
calculate_cross_border_tax	VAT/GST, import duty, landed cost	Model knowledge, as structured JSON → a conservative 15% fallback is applied and flagged
validate_fx_hedge	Quoted rate vs market, ±5% band	**Live Frankfurter API** (<0.1 s) → a labelled model estimate is used → FX WARNING
categorize_expense	CapEx vs OpEx, depreciation, sign-off flags	IFRS rules applied by the model → a labelled keyword heuristic is used as fallback

Prompt design as policy design. The same four-part structure is followed by every specialist prompt: **Role** → **Responsibilities** → **Decision framework** → **Output format**. Prompts can be read and amended by policy owners like an operating procedure, without code being touched.

Guardrails: airport security around every agent

The agent's think-act loop is wrapped by middleware, without its prompt or tools being changed. The same checks are applied, in order, to every request and every tool call. Privacy, cost control, resilience and auditability are therefore built into the architecture rather than left to each prompt.



AUDIT LOG EXTRACT (ILLUSTRATIVE)

```
{"event":"Tool call started","tool":"validate_fx_hedge","args":{"currency_pair":"EUR_JPY","rate_used":163.5}}
{"event":"Tool call finished","tool":"validate_fx_hedge","duration_ms":105,"preview":"FX ALERT: ... 5% limit"}
```

When an auditor asks why a purchase was held, the evidence has already been recorded.

Figure 7. Middleware stack shared by all three specialists. Limits and retries are set in configuration, not in code.

Governance and risk controls

Risk	Control in the solution
A wrong call is made by the AI	Human sign-off remains mandatory. The decision is governed by explicit keyword rules. Repeatable output is obtained with temperature 0, and review is speeded up by fixed report layouts.
Policy is hallucinated	RAG answers are drawn only from retrieved company text, the source file is named, and "I don't know" is returned when the answer is absent.
Personal data is leaked	E-mail addresses and card numbers are redacted **before** text reaches the model. Only fictional documents are used in the demo. Secrets are never stored in code or images.
Cost runs away or loops occur	Model calls (8) and tool calls (6) are capped per phase. Cloud Run is limited to 3 instances. Budget alerts and a clean-up procedure are provided.
Services fail or are flaky	Retries with backoff are applied. Errors are caught by each tool and downgraded to "manual review required". A missing setting is reported as a clear HTTP 503 rather than a crash.
No audit trail exists	Every phase and tool call is written to JSON logs, which are kept in Cloud Storage and Cloud Logging. The full memo serves as the decision record.

5 · FUNCTIONAL VIEW OF THE CODE

The codebase as an organisation chart

The code is organised on a "one file, one job" basis, in the way a well-run kitchen is divided into stations. A single responsibility is assigned to each functional module, and the modules it may call are clearly defined.

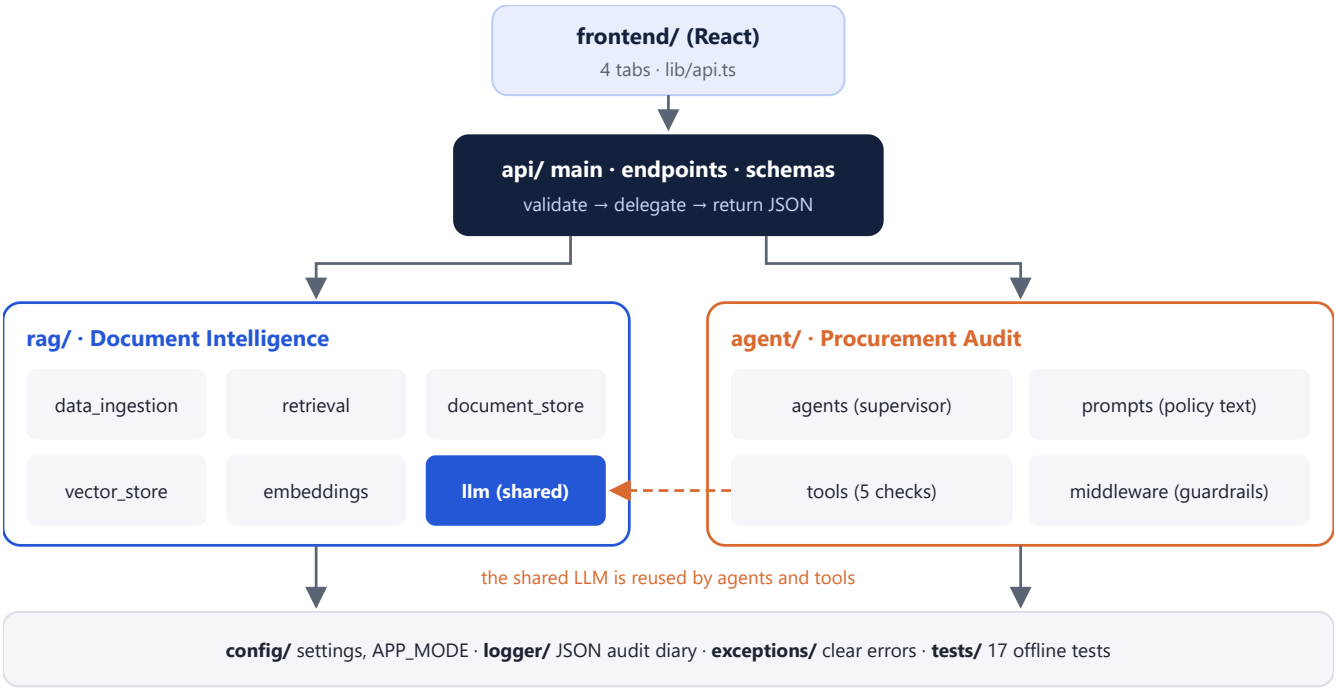


Figure 8. Functional module map and dependency direction.

Module	Single responsibility	Input → output
frontend/ · lib/api.ts	User interface; all backend calls are owned by one file	User clicks → HTTP requests; JSON → rendered results
api/endpoints.py	Thin routes: input is validated, work is delegated, errors are translated	HTTP JSON / file upload → function call → JSON reply
api/schemas.py	The contracts ("order forms")	AuditRequest → AuditResponse (4 texts); QueryRequest → QueryResponse
rag/data_ingestion.py	Documents are brought in	PDF path → pages → chunks → stored vectors; page and chunk counts returned
rag/retrieval.py	Answers are produced	Question + strategy → retrieved chunks → grounded answer
rag/vector_store.py, embeddings.py, document_store.py	Storage adapters selected by APP_MODE (strategy pattern)	The same interface is exposed whether local (Chroma, disk) or cloud (Vertex, GCS)
agent/agents.py	The audit committee is orchestrated	Request text → 3 specialist reports → CFO memo (dict of 4)
agent/tools.py, prompts.py, middleware.py	Instruments, job descriptions and guardrails	Tool arguments → check result; briefs; a wrapped, logged agent loop
config/, logger/, exceptions/	Foundation used by all modules	.env → typed settings; events → JSON log lines; faults → 503/422 replies

Journey 1: "ask the policy", from click to answer

The path of a question through the modules and back is shown below. The same pattern (thin route, specialised module, shared model, logged result) is repeated across the whole system.

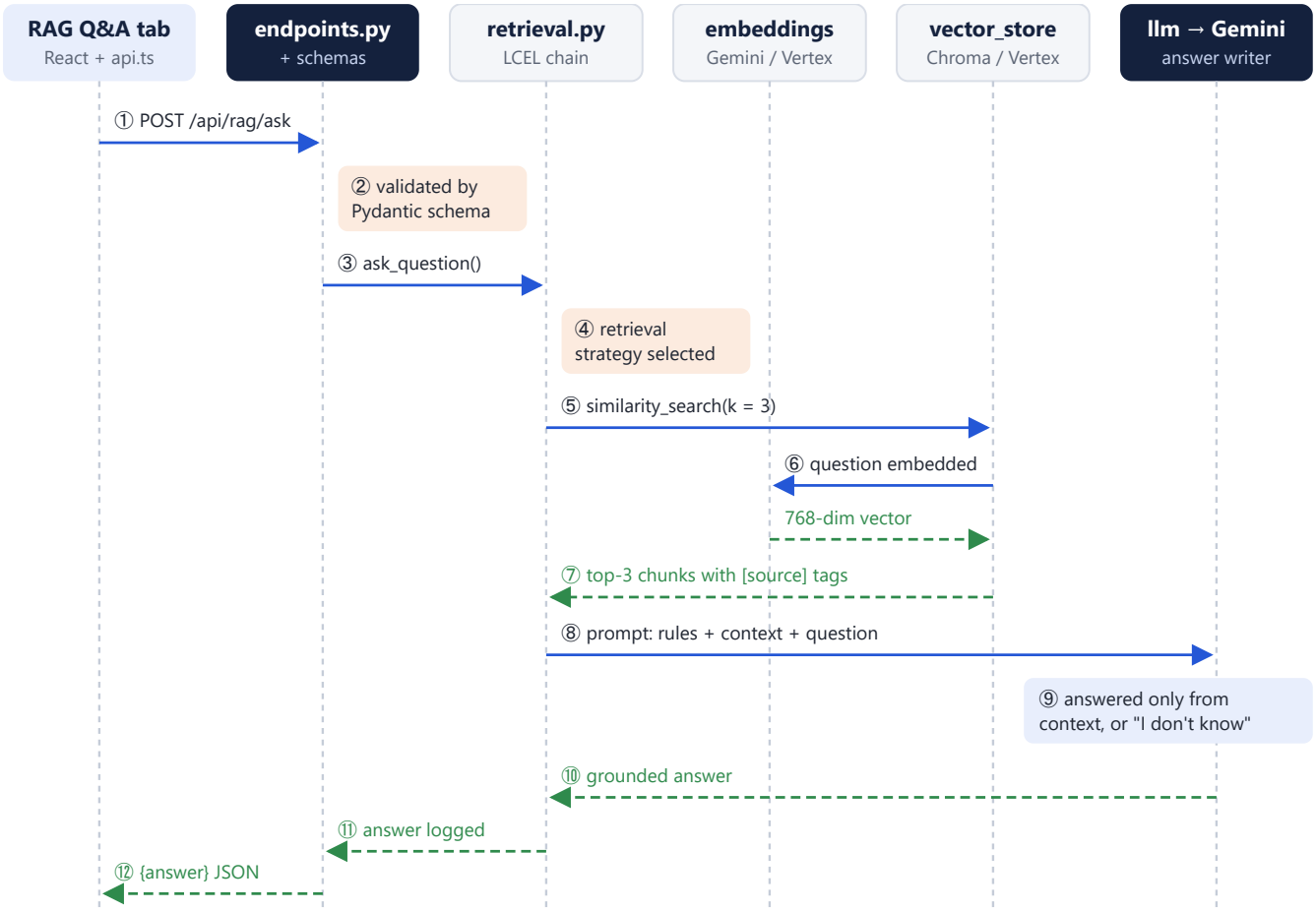


Figure 9. Sequence for a policy question (answer shown in about 4 s). Solid blue lines are calls; dashed green lines are returned data.

Supporting flow: a policy document is published

1. PDFs are sent from the **Index Documents tab** to `POST /api/rag/upload`. Non-PDF files are skipped with a reason.
2. The original is kept by **document_store** (disk or Cloud Storage). Indexing is continued even if this copy fails.
3. **data_ingestion** is run in a worker thread so that the server stays responsive: pages → chunks → embeddings → vector store.

Each file's status is returned with page and chunk counts; image-only PDFs are reported as "skipped: no extractable text". All stored documents can be re-indexed in bulk through `POST /api/rag/ingest`, where the same ingestion function is reused, so behaviour is defined once.

Journey 2: "check this purchase", request to memo

The same request is passed by the supervisor to each specialist in turn. Each think → tool → observe loop is run inside the guardrails, and only the finished reports are passed to the CFO step.

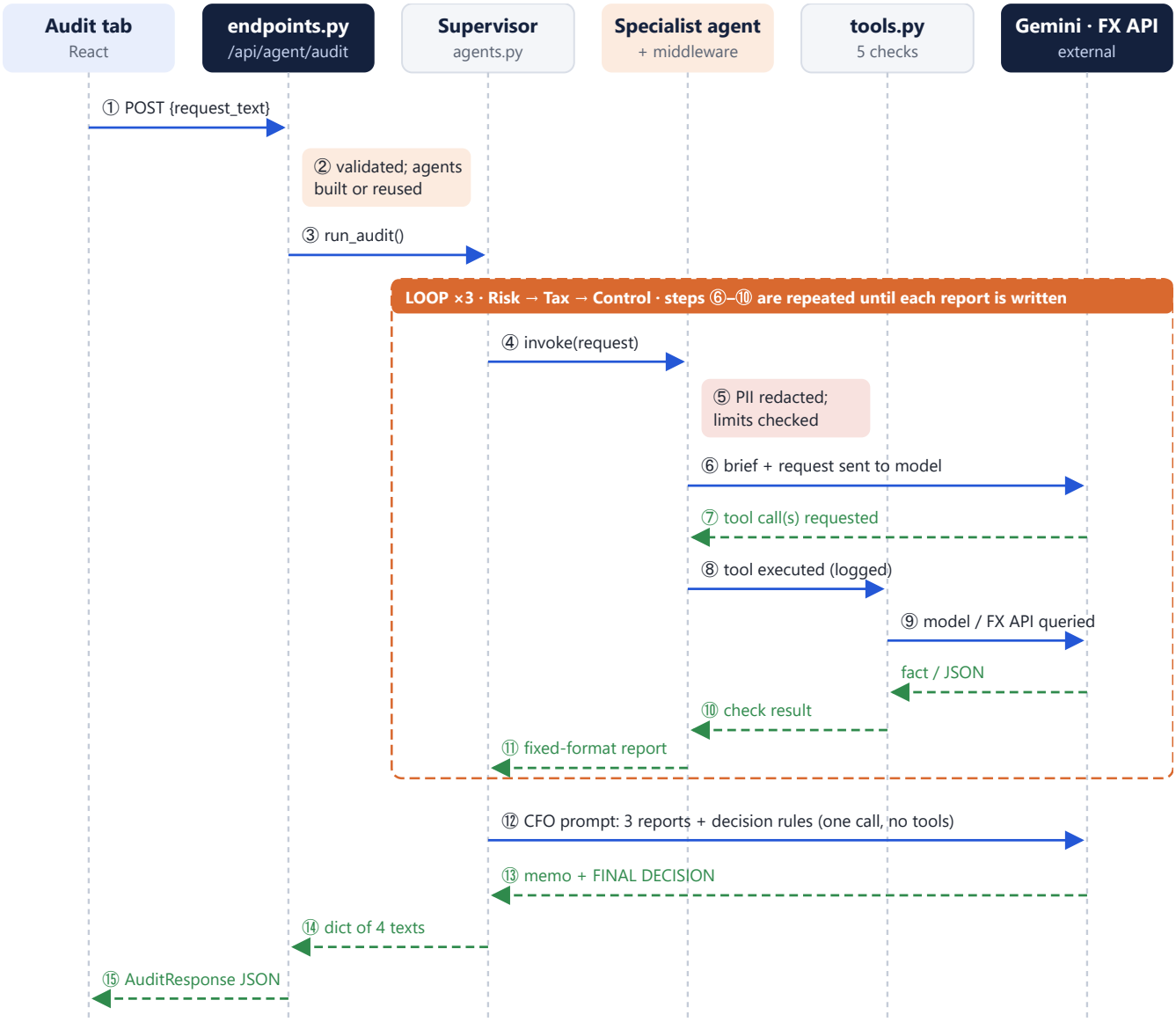


Figure 10. Audit sequence (results shown after about 70–150 s). Steps ④–⑪ are repeated for each specialist; only the three finished reports are seen by the CFO step.

Why the hand-off is reliable

A **fixed layout** is required from each specialist (e.g. `OVERALL RISK VERDICT`, `TREASURY VERDICT`), and fixed keywords (`RED ALERT`, `FX ALERT`) are emitted by the tools. Because the CFO rules depend on these words, the reports act as a contract between modules.

Observed agent behaviour

Both Tax tools were called at the same time. In the sanctioned case, the credit check was **skipped after the RED ALERT**, as instructed by the brief. The live FX check was completed in under 0.1 s; model-backed tools took 4–13 s each.

Built to be trusted, run and afforded

17
automated tests are passed. No accounts or internet are needed, because the real models are replaced by fakes.

~1.9k
lines of Python are organised into 5 functional packages, alongside a React/TypeScript UI

7
REST endpoints are exposed, with auto-generated interactive API documentation (/docs)

1
setting is used to switch between laptop and cloud, with no code change

Two levels of quality assurance

What is tested	How it is tested	Why it matters
Controlled parts (wiring)	A full local run (PDF → Chroma → answer) and an agent run through the middleware are executed with fake AI models	Fast, free and repeatable; safe to run on every change.
Model-written parts (answers)	Known-answer questions are asked against the sample documents, and three predicted audit scenarios are run (approve, hold, reject)	Business correctness is proven; a regression suite can be formed.
Configuration failures	When a key or cloud setting is missing, HTTP 503 is returned with exactly what must be set	Problems are resolved in minutes, not hours.

Operability

- Mode, uptime, models, vector store and storage location are shown on the **System Status tab** (`/api/status`). No cloud calls are made, so the page remains available even when the cloud is misconfigured.
- **Structured JSON logs** are written to the terminal and to files, with a `severity` field for Google Cloud Logging. In cloud mode they are flushed to Cloud Storage on shutdown, because container disks are temporary.
- **Dependency versions are pinned**, so behaviour is kept stable while AI libraries change. Model names are held as settings, so a retired model can be replaced without a code change.
- **Lazy initialisation** is used: models and connections are created on first use, so the service is started and passes health checks before everything is configured.

Cost model and controls

Cost item	How it is billed	Control in place
Gemini model calls	Per token (a free tier is available)	Temperature 0, focused prompts, per-phase caps on model calls (8) and tool calls (6)
Vertex AI Vector Search	**Hourly while deployed**, even when idle; this is the main cost	Pilots are run locally on Chroma first; a budget alert and a documented tear-down are provided
Cloud Run	Only while requests are served (scales to zero)	Cost and abuse are capped at 3 instances
Storage, Build, Secrets	Cents, or within free allowances	No control is required

> **Commercial framing:** value can be proven by a business unit in local mode for little more than model usage. Spending on managed infrastructure is committed only after a measured gain against the baseline has been shown.

Limits are stated openly; the path to production is defined

This is a working reference implementation built on fictional data; it is not yet a compliance system of record. By stating the gap precisely, the next step can be supported by risk, audit and IT leaders.

Current limit	Why it matters → planned remedy
4 of 5 audit checks rely on the model's general knowledge	Results are not authoritative (0% vs 2.2% duty) → real sanctions, credit-agency and tariff sources, and the company's own RAG index, are to be connected
No sign-in; CORS left open in development	The app can be used by anyone with the URL → SSO, role-based access and an approval workflow are to be added
Upload history held in memory	It is lost on restart → a database is to be added for history, audits and memos
AI wording varies between runs	Every decision is reviewed by a person → structured outputs and keyword checks in code are to be introduced
Only text-based PDFs supported	Scans are skipped → an OCR step is to be added
Specialists run one after another	Audits take 70–150 s → specialists are to be run in parallel, with progress streamed to the UI
Broad CI permissions and a stored key file	A security risk is created → keyless Workload Identity Federation and least-privilege roles are to be adopted

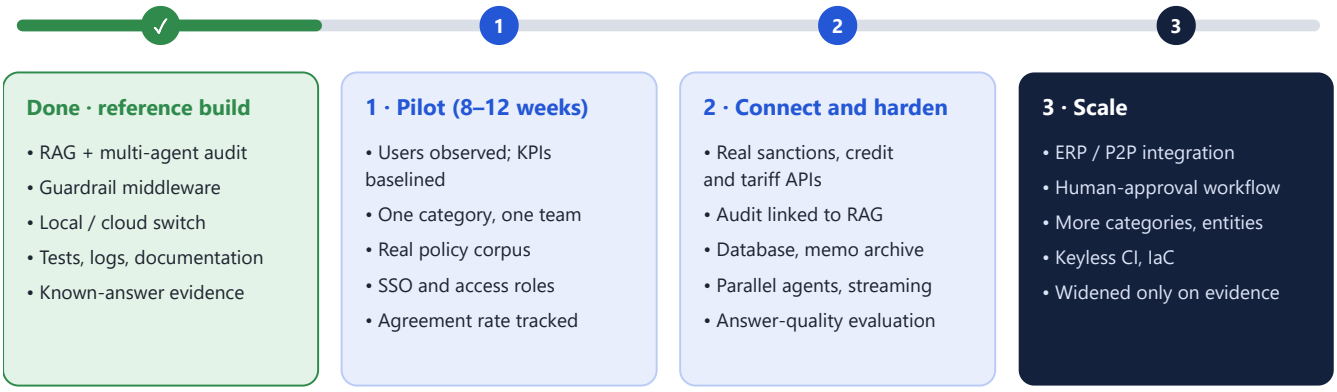


Figure 11. Phased roadmap: the scope is started narrow, measured against the baseline, then widened on evidence. Durations are indicative.

8 · KEY TAKEAWAYS

What the project demonstrates

The technology is treated as the means. The capabilities evidenced by the project, and their relevance to wider business transformation, are summarised below.

1 · Problem before technology

The process was analysed first: four pains were identified, the cost of a miss was described, and cheaper fixes (reminders, a shared folder, a generic chatbot, a rules engine) were ruled out before AI was chosen. Every component is mapped to a business need.

2 · Value that can be proven

Success was defined as a measured gain against a baseline. Known-answer tests and predicted-outcome scenarios were built, and the observed results were published, including where the system was found to be wrong.

3 · Governance designed in

Decision rights are retained by people. Outcomes are governed by written rules. Privacy, cost caps and audit logs are part of the architecture. In this way, the trust of Finance, Risk and Audit can be earned.

4 · Pragmatic trade-offs

Readable orchestration was preferred to clever autonomy, specialists to a super-agent, and a laptop pilot to early cloud spend. A cost and a reversal path are stated for each choice.

5 · Candour about limits

What is simulated and what is real are stated explicitly. A surfaced 0% vs 2.2% discrepancy was made the top roadmap item rather than left as a hidden risk.

6 · Adoption and handover

Policy is held in readable prompts that can be changed by business owners. Documentation, tests, a status page and logs are supplied, so the system can be run, changed and switched off by the receiving team.

Anticipated stakeholder questions

Question	Short answer
"Can payments be approved by it?"	Not on its own, by design. A rule-based recommendation is prepared with evidence, and approval is given by a named person. Trust is built through the measured decision-agreement rate.
"What is the ROI?"	The levers are cycle time, rework, consistency and audit effort. They are to be baselined in the pilot and the measured gain reported. No figure is promised before measurement.
"What is the biggest risk?"	Confidently wrong checks caused by non-authoritative data. This is mitigated by connecting authoritative sources first, keeping human sign-off and logging everything.
"How can this be scaled across the group?"	The same pattern can be applied to any policy-heavy approval: expenses, contracts, credit and onboarding. Specialists and documents are added; new platforms are not required.
"Should it be built or bought?"	Vendors are kept swappable by the layered design: model (Gemini, Claude or GPT), vector store and cloud. Commodity parts can be bought, while the policy logic and the data are owned.

APPENDIX

Technology stack and glossary

A. Technology stack

Layer	Technology
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn/ui
API	FastAPI, Pydantic v2, Uvicorn
AI framework	LangChain v1 (LCEL, create_agent on LangGraph, middleware)
LLM	Google Gemini (flash model)
Embeddings	gemini-embedding-001 / text-embedding-005 (768-dim)
Vector DB	Chroma (local) / Vertex AI Vector Search (cloud)
Storage	Local disk / Google Cloud Storage
Live data	Frankfurter FX-rate API
Hosting	Docker, Google Cloud Run
Security	Secret Manager, PII middleware
CI/CD	GitHub Actions + gcloud (idempotent)
Observability	structlog JSON logs, Cloud Logging
Testing	pytest (17 offline tests), Vitest

B. Sample documents (fictional)

- Aldermoor annual report FY2024
- Aldermoor procurement policy
- Aldermoor accounting policy (IFRS)
- EU tariff schedule excerpt

C. Glossary for non-technical readers

Term	Plain meaning
LLM	An AI model by which text is read and written (here, Gemini).
RAG	The right document pages are looked up first, and the answer is written from them: an "open-book exam".
Embedding	A list of numbers in which the meaning of a text is captured, like GPS coordinates for ideas.
Vector database	A library catalogue organised by meaning rather than by title.
Agent	An AI by which tools (look-ups, calculators) can be used before answering.
Tool	A small function that can be called by an agent, such as "check the exchange rate".
Prompt	The written brief by which the AI's role, rules and output format are set.
Middleware	Checks wrapped around every AI step: privacy, limits, retries and logging.
API	The "waiter" between the screen and the logic: one web address per job.
Hallucination	A fluent but false AI answer; it is reduced by grounding and "I don't know".
CapEx / OpEx	Spending on a lasting asset vs a running cost; the balance sheet and P&L are affected differently.
FX hedge band	The allowed gap (±5%) between a quoted exchange rate and the market rate.

Disclaimer. Aldermoor Industries, its people, documents and figures are fictional and were created for this project. Audit "checks" other than FX are performed from AI general knowledge for demonstration. This is a reference implementation, not a compliance tool. Observed timings and results were taken from test runs in October 2026.